Toward Explainable Agentic Cyber Threat Intelligence: Semantic Smart Contracts, Argumentation Ranking and Runtime Observability

Alessandro Costa
M.Sc. Computer Engineering

Abstract

Modern software supply chains increasingly require proactive and explainable cybersecurity mechanisms capable of integrating runtime observability, semantic reasoning, collaborative intelligence, and verifiable decision-making. Traditional Cyber Threat Intelligence (CTI) platforms mainly provide static vulnerability reporting and reactive alerting, often lacking contextual reasoning, autonomous mitigation orchestration, and auditable strategic validation.

This thesis proposal introduces an agentic CTI architecture integrating OpenTelemetry runtime observability, collaborative multi-agent reasoning, semantic smart contracts, argumentation-based strategy ranking, provenance-aware blockchain traceability, and OWL-based CTI Knowledge Graphs generated from standard vulnerability datasets such as CVE, OSV, and SBOM sources.

The proposed framework aims to transform CTI from a passive vulnerability analysis process into an autonomous and explainable ecosystem capable of observing runtime evidence, correlating semantic cybersecurity knowledge, validating mitigation strategies, and supporting deterministic security decision-making through blockchain-based reasoning mechanisms.

The thesis investigates the integration of semantic technologies, collaborative agents, argumentation frameworks, and distributed ledger infrastructures to support explainable and auditable cybersecurity orchestration in modern software supply chains.

1 Introduction

Modern software supply chains require proactive, explainable and verifiable cybersecurity mechanisms capable of operating on runtime evidence and dynamic threat intelligence. Traditional vulnerability management systems provide static findings but lack collaborative reasoning, contextual awareness and deterministic decision-making.

This proposal introduces an end-to-end Cyber Threat Intelligence (CTI) architecture integrating:

- OpenTelemetry runtime observability;
- collaborative multi-agent reasoning;
- semantic smart contracts;
- argumentation-based strategy ranking;
- provenance-aware blockchain traceability;
- OWL CTI Knowledge Graph generation from standard vulnerability datasets.

The architecture transforms CTI from passive vulnerability reporting into an autonomous ecosystem capable of observing, reasoning, validating and selecting mitigation strategies in an explainable and auditable way.

2 Architecture Overview

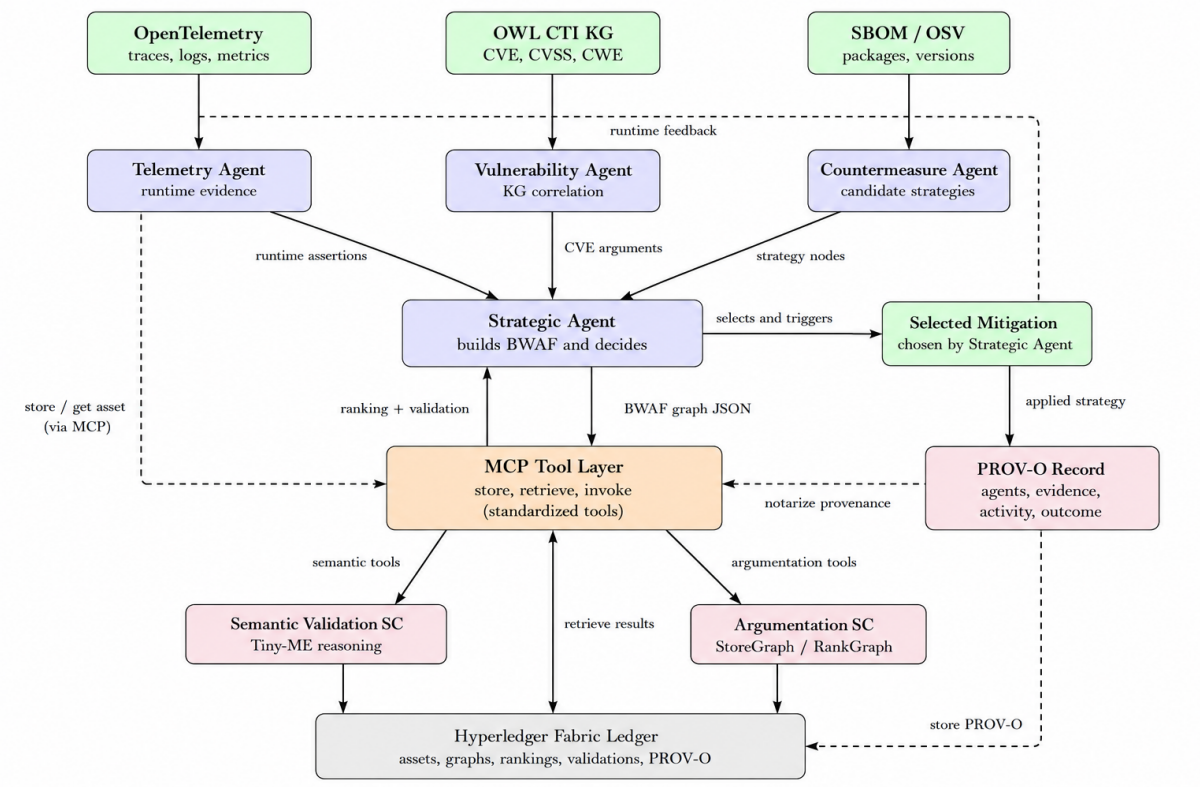


Figure 1: Agentic CTI workflow with MCP-mediated smart contract invocation. The Argumentation Smart Contract computes the ranking of candidate mitigation strategies, while the Semantic Validation Smart Contract provides semantic validation and explainable reasoning through Tiny-ME. The Strategic Agent receives ranking and explanation results, selects the final mitigation, and records the applied strategy together with PROV-O provenance metadata on Hyperledger Fabric.

Figure 1 shows the proposed end-to-end workflow. OpenTelemetry provides runtime evidence, while the OWL CTI Knowledge Graph and SBOM/OSV sources provide structured vulnerability and dependency knowledge. Specialized agents transform these inputs into semantic arguments and candidate mitigation strategies.

The Strategic Agent aggregates the arguments, builds the BWAF graph and invokes blockchain tools through the MCP Tool Layer. MCP acts as the standard interface between the agent community and Hyperledger Fabric smart contracts, exposing operations for asset storage and retrieval, argumentation ranking, semantic validation and provenance notarization.

The Argumentation Smart Contract ranks candidate strategies through the BWAF reasoning process, while the Semantic Validation Smart Contract performs ontology-based semantic reasoning through Tiny-ME, validating the selected strategy, detecting semantic conflicts, and generating explainable justifications through abduction, bonus and contraction reasoning services.

The Strategic Agent receives both the ranking results and the semantic explanations through MCP, selects and triggers the final mitigation strategy, and finally records the applied action together with its provenance metadata as a PROV-O record on the Hyperledger Fabric ledger.

3 Integration with the CTI Knowledge Graph

The proposed architecture integrates an OWL CTI Knowledge Graph generated from standard vulnerability records such as CVE, OSV and SBOM datasets.

The semantic pipeline transforms JSON vulnerability records into ontology-aligned OWL representations. The resulting Knowledge Graph contains:

- vulnerability individuals;
- software products;
- affected versions;
- CVSS metrics;
- CWE weakness categories;
- semantic relationships among threats and assets.

The generated Knowledge Graph acts as the semantic source of truth for the agentic architecture.

The Vulnerability Agent queries the OWL CTI Knowledge Graph and correlates KG individuals with runtime software assets observed through OpenTelemetry.

For example, if the Telemetry Agent observes a vulnerable package at runtime, the Vulnerability Agent searches the KG to retrieve:

- matching CVE entries;
- CVSS severity;
- associated CWE classes;
- affected products and versions;
- related threat metadata.

The resulting semantic assertions become arguments for the reasoning pipeline.

The CTI Knowledge Graph is not used as a passive vulnerability repository, but as a semantic substrate enabling agents to generate contextual arguments, threat relations and mitigation hypotheses.

4 Collaborative Agent Community

The architecture models a community of collaborative agents.

4.1 Telemetry Agent

The Telemetry Agent consumes OpenTelemetry traces, metrics and logs.

Example input:

```
{
  "service.name": "payment-api",
  "trace_id": "trace-abc",
  "package.name": "log4j-core",
  "package.version": "2.14.1",
  "endpoint.exposed": true
}
```

The Telemetry Agent generates semantic runtime evidence.

Example output:

```
{
  "type": "runtime_evidence",
  "asset": "payment-api",
  "evidence": [
    "uses log4j-core 2.14.1",
    "endpoint exposed",
    "runtime deserialization observed"
  ]
}
```

4.2 Vulnerability Agent

The Vulnerability Agent queries the OWL CTI Knowledge Graph generated from standard CVE records and correlates vulnerability individuals with runtime software assets observed through OpenTelemetry.

Example:

```
{
  "id": "A_CVE_LOG4J",
  "claim": "payment-api is vulnerable to critical RCE",
  "cvss": 10.0,
  "cwe": "CWE-502"
}
```

4.3 Threat Agent

The Threat Agent enriches findings with:

- MITRE ATT&CK techniques;
- CWE classes;
- exploitability;
- threat severity;
- attack patterns.

4.4 Countermeasure Agent

The Countermeasure Agent generates candidate mitigation strategies.

Example output:

```
{
  "strategies": [
    {"id":"S_UPGRADE"},
    {"id":"S_ISOLATE"},
    {"id":"S_BLOCK_DEPLOY"},
    {"id":"S_MONITOR"}
  ]
}
```

4.5 Strategic Agent

The Strategic Agent orchestrates the reasoning process.

Its responsibilities are:

- aggregating arguments produced by all agents;
- constructing the BWAF graph;
- assigning attack/support relations;
- serializing the graph in JSON format;
- invoking blockchain smart contracts through MCP.

The Strategic Agent does not autonomously compute the ranking itself. Instead, it constructs the BWAF graph and delegates the ranking process to the Argumentation Smart Contract. After receiving ranking and semantic validation results through MCP, it selects and triggers the final mitigation.

5 Argumentation Framework

The proposal integrates the argumentation ranking approach implemented in the Argumentation Smart Contract.

5.1 Free Nodes

Free nodes represent semantic evidence and contextual assertions.

Examples include:

- runtime traces;
- detected CVEs;
- vulnerable packages;
- CVE individuals retrieved from the CTI Knowledge Graph;
- CVSS severity scores;
- CWE weakness categories;
- exposed endpoints;
- compliance constraints.

These nodes act as contextual axioms.

5.2 Connected Nodes

Connected nodes represent mitigation strategies generated by the Countermeasure Agent.

Examples include:

- package upgrade;
- service isolation;
- deployment blocking;
- passive monitoring;
- credential rotation.

5.3 BWAF Construction

The Strategic Agent transforms semantic evidence into a Bipolar Weighted Argumentation Framework (BWAF).

Example:

```
{
  "nodes": [
    {"id":"A_CVE","type":"vulnerability"},
    {"id":"A_RUNTIME","type":"evidence"},
    {"id":"S_UPGRADE","type":"strategy"},
    {"id":"S_ISOLATE","type":"strategy"}
  ],
  "edges": [
    {"from":"A_CVE","to":"S_UPGRADE"},
    {"from":"A_RUNTIME","to":"S_ISOLATE"}
  ]
}
```

Free nodes represent contextual evidence, while connected nodes represent candidate mitigation strategies evaluated by the ranking algorithm.

6 Smart Contracts and MCP Integration

Smart contracts are not used merely as storage components, but as deterministic and verifiable reasoning authorities operating on-chain.

The architecture exposes semantic smart contracts through MCP-compatible tools.

6.1 Argumentation Smart Contract

The Argumentation Smart Contract exposes:

- `StoreGraph(graphKey, graphJson)`
- `RankGraph(graphKey)`

The Strategic Agent serializes the BWAF graph and submits it through MCP.

Example MCP invocation:

```
{
  "tool": "argumentation.rankGraph",
  "arguments": {
    "graphKey": "payment-api-case"
  }
}
```

Example output:

```
{
  "winner": "S_UPGRADE",
  "score": 0.91
}
```

6.2 Semantic Smart Contract

After ranking, the selected strategy is validated semantically.

The semantic smart contract performs:

- concept abduction;

- concept bonus;
- concept contraction.

Example invocation:

```
{
  "tool": "semantic.validateCountermeasure",
  "arguments": {
    "asset": "payment-api",
    "countermeasure": "S_UPGRADE"
  }
}
```

7 Semantic Reasoning

Semantic reasoning is implemented through Tiny-ME integrated on-chain via CGO.

The reasoning engine supports:

- semantic matchmaking;
- explanation generation;
- conflict detection;
- mitigation validation.

This transforms smart contracts into autonomous semantic agents capable of explainable reasoning.

8 PROV-O Integration

The architecture integrates PROV-O to model provenance and traceability.

8.1 Entities

Examples:

- vulnerabilities;
- runtime traces;
- SBOM artifacts;
- findings;
- mitigation strategies.

8.2 Activities

Examples:

- telemetry analysis;
- vulnerability matching;
- argumentation ranking;
- semantic validation.

8.3 Agents

Examples:

- Telemetry Agent;
- Vulnerability Agent;
- Countermeasure Agent;
- Strategic Agent;
- semantic smart contracts.

The provenance layer records not only the final mitigation outcome, but also the complete decision lineage, including: (i) runtime evidence, (ii) semantic arguments, (iii) BWAF ranking activities, (iv) semantic validation results, (v) involved agents, and (vi) triggered mitigation actions.

Example provenance representation:

```
{
  "entity": "finding-payment-api",
  "wasDerivedFrom": [
    "trace-abc",
    "package-log4j"
  ],
  "wasAttributedTo": [
    "TelemetryAgent",
    "VulnerabilityAgent"
  ],
  "selectedStrategy": "S_UPGRADE"
}
```

9 Software Baselines and Thesis Contribution

The implementation is based on two pre-existing software projects: `agent-based-cti-argumentation-pipelin` and `semantic-cti-ledger-argumentation`. These projects must be treated as technical baselines rather than as results of the thesis. Their role is to provide reusable components for telemetry processing, CTI correlation, semantic knowledge representation, blockchain interaction, argumentation ranking, and semantic reasoning.

The original contribution of the thesis therefore does not consist in merely executing or documenting the two repositories. It consists in selecting, adapting, extending, and integrating their components into a coherent end-to-end architecture, and in experimentally validating the resulting workflow.

The thesis must explicitly distinguish:

- functionality already available in the original repositories;
- functionality modified or extended during the thesis;
- newly implemented components;
- experimental activities and results produced specifically for the thesis;
- features discussed at the architectural level but not implemented in the prototype.

This distinction is essential both for scientific reproducibility and for making the student's individual contribution verifiable.

9.1 Role of the Agent-Based CTI Argumentation Pipeline

The project `agent-based-cti-argumentation-pipeline` is used as the starting point for the acquisition, processing, and orchestration layer of the architecture. Its reusable functions include:

- ingestion and preprocessing of OpenTelemetry data;
- extraction of software assets and package versions from runtime logs;
- product matching against a vulnerability index derived from the CTI Knowledge Graph;
- verification of vulnerable software versions;
- generation of OWL assertions describing observed assets;
- construction of an initial argumentation graph;
- interaction with Hyperledger Fabric through an MCP-to-Fablo bridge.

The relevant implementation areas include the main pipeline, the semantic asset processing modules, the ledger interaction layer, and the MCP server exposing blockchain operations.

However, the baseline repository must not automatically be described as a fully implemented collaborative multi-agent system. If its execution model remains a sequence of procedural modules, the thesis must either implement explicit agent components or use more precise terminology such as *agent-inspired pipeline* or *logical agent decomposition*.

For each agent introduced in Section 4, the implementation must therefore specify:

- input data;
- output data;
- internal responsibility;
- message format;
- invocation mechanism;
- relationship with the other agents;
- whether the component is autonomous, reactive, or simply procedural.

At minimum, the prototype should provide a clear implementation or formal mapping for the Telemetry Agent, Vulnerability Agent, Countermeasure Agent, and Strategic Agent.

9.2 Role of the Semantic CTI Ledger and Argumentation Project

The project `semantic-cti-ledger-argumentation` is used as the blockchain and reasoning backend. Its reusable components include:

- a Hyperledger Fabric network configured through Fablo;
- semantic chaincode integrating Tiny-ME through CGO;
- reasoning services for concept abduction, bonus, contraction, discovery, and explanation;
- argumentation ranking chaincode;
- ledger persistence of graphs, annotated graphs, and ranking results;

- determinism and persistence tests;
- monitoring infrastructure;
- Hyperledger Caliper benchmarking resources.

Only the components directly related to CTI reasoning, countermeasure validation, argumentation ranking, provenance, and experimental evaluation should be included in the thesis workflow. Functions related to generic negotiation, service discovery, or agreement management should be considered legacy or out of scope unless they are explicitly reused in the CTI case study.

9.3 Integration Objective

The expected result is a single integrated pipeline implementing the following sequence:

1. acquisition of OpenTelemetry runtime evidence;
2. extraction of software assets and versions;
3. correlation with vulnerabilities represented in the CTI Knowledge Graph;
4. generation of contextual semantic evidence;
5. production or retrieval of candidate mitigation strategies;
6. construction of a Bipolar Weighted Argumentation Framework;
7. storage of the graph on the ledger through MCP;
8. execution of on-chain argumentation ranking;
9. retrieval of the ranking result and annotated graph;
10. semantic validation of the highest-ranked strategy;
11. selection or recommendation of the final mitigation;
12. recording of the complete decision provenance.

The integration should expose the complete MCP interaction cycle. At minimum, the argumentation workflow should include operations equivalent to:

$$\texttt{store_graph} \rightarrow \texttt{rank_graph} \rightarrow \texttt{get_rank_result} \rightarrow \texttt{get_annotated_graph}.$$

The MCP layer should also expose domain-level semantic reasoning tools. A tool such as `validate_countermeasure` is preferable to requiring the Strategic Agent to manually coordinate low-level blockchain functions.

10 Formalization of the Argumentation Graph

The argumentation graph is a central research component and must be described beyond its serialization format.

The prototype must define the semantics of:

- free nodes representing contextual evidence;
- connected nodes representing candidate strategies;

- support relations;
- attack relations;
- node weights;
- edge weights;
- graph direction;
- ranking interpretation.

Free nodes may represent runtime traces, CVE assertions, CVSS values, CWE categories, exposed endpoints, dependency information, asset criticality, and compliance constraints. Connected nodes represent mitigation strategies such as software upgrade, service isolation, deployment blocking, passive monitoring, or credential rotation.

The construction process must explain how semantic assertions are transformed into attack and support relations. Edges between strategies must not be generated only to satisfy the input format of the smart contract. Every relation should have a documented methodological or semantic justification.

For example, a critical CVE affecting an exposed runtime service may support both package upgrade and service isolation. A strict availability constraint may attack a strategy requiring immediate shutdown. A compatibility constraint may attack a package upgrade if the target version conflicts with another dependency.

The thesis should document:

- the rules used to create nodes and edges;
- the source of each weight;
- the normalization procedure;
- the handling of missing evidence;
- the treatment of conflicting evidence;
- the conditions under which a strategy is considered the winner.

11 Countermeasure Generation and Representation

The thesis must accurately describe how candidate countermeasures are obtained. Depending on the implemented prototype, strategies may be:

- retrieved from semantic relations already stored in the Knowledge Graph;
- generated through deterministic rules;
- proposed by an LLM-based agent;
- produced through a hybrid combination of retrieval and generation.

A component should not be described as a generative Countermeasure Agent if it only performs a query over pre-existing `hasSolutions` or `hasWorkarounds` relations.

Each strategy should include, when available:

- a unique identifier;

- a human-readable description;
- the vulnerability or asset it addresses;
- technical prerequisites;
- expected security benefit;
- operational impact;
- reversibility;
- execution constraints;
- provenance of the recommendation.

This representation allows the argumentation layer to compare strategies using both security and operational evidence.

12 Semantic Validation of Ranked Strategies

The highest-ranked strategy must not be accepted automatically. Ranking and semantic validation are separate stages with different responsibilities.

The Argumentation Smart Contract orders candidate strategies according to the structure and weights of the BWAF. The Semantic Validation Smart Contract then checks whether the selected strategy is semantically applicable to the specific asset and operational context.

The validation stage should examine:

- compatibility with the asset type;
- consistency with the CTI ontology;
- missing prerequisites;
- conflicts with operational constraints;
- retained and sacrificed requirements;
- semantic distance between the offered countermeasure and the required mitigation;
- explanations produced through abduction, bonus, and contraction.

A possible outcome is that the highest-ranked strategy is rejected or conditionally accepted because it violates a semantic or operational constraint. In that case, the Strategic Agent should evaluate the next candidate or report the missing conditions required to make the strategy applicable.

The resulting explanation should make clear:

- why the strategy was ranked highly;
- why it is semantically valid or invalid;
- which evidence supported the decision;
- which conflicts or missing conditions were detected;
- why the final strategy was preferred to the alternatives.

13 Provenance Model and PROV-O Recording

The provenance layer must permit reconstruction of the complete decision lineage, not only the final selected strategy.

A minimal PROV-O representation should include:

- **Entities**: runtime logs, traces, software assets, CVE records, OWL assertions, argumentation graphs, ranking results, candidate strategies, validation reports, and final decisions;
- **Activities**: telemetry extraction, vulnerability matching, semantic enrichment, graph construction, ranking, semantic validation, and mitigation selection;
- **Agents**: software agents, human operators, MCP services, and smart contracts;
- **Relations**: `used`, `wasGeneratedBy`, `wasDerivedFrom`, `wasAssociatedWith`, and `wasAttributedTo`.

The ledger should enable an auditor to answer at least the following questions:

- Which runtime evidence initiated the analysis?
- Which CTI records were matched?
- Which agents and contracts participated?
- How was the argumentation graph generated?
- Which strategies were ranked?
- Why was one strategy selected?
- Was the strategy semantically validated?
- Which action was finally recommended or executed?

14 Implementation Scope and Terminological Constraints

The thesis must maintain strict correspondence between architectural claims and the actual prototype.

The following constraints apply:

- the system should not be described as real-time if it processes only batch files;
- the term multi-agent system should be used only if agents and their interactions are explicitly implemented or rigorously modeled;
- OSV and SBOM integration should not be claimed unless they are actually included in the implemented and evaluated pipeline;
- autonomous mitigation should not be claimed if the system only produces a recommendation;
- PROV-O integration should not be claimed until provenance records are actually generated;
- an LLM-based capability should not be claimed if the corresponding module uses only deterministic retrieval or hard-coded rules.

A simulated or dry-run mitigation is acceptable for the thesis. In that case, the result must be described as a selected or recommended mitigation rather than as an automatically executed change on a production system.

15 Experimental Evaluation

The evaluation must measure both component-level performance and end-to-end behavior.

15.1 Extraction and Vulnerability Matching

A manually annotated sample should be used to evaluate the extraction of software product, vendor, and version information. The thesis should report:

- precision;
- recall;
- F1-score;
- product matching accuracy;
- vulnerable-version classification accuracy;
- error categories and representative failure cases.

The dataset should contain at least one vulnerable and one non-vulnerable case, including ambiguous or incomplete runtime evidence where possible.

15.2 Argumentation Ranking

The ranking evaluation should include:

- determinism when nodes and edges are provided in different orders;
- persistence of graphs and results on the ledger;
- consistency of the selected winner across repeated executions;
- sensitivity to node and edge weights;
- behavior on graphs of increasing size;
- qualitative analysis of generated explanations.

15.3 Semantic Validation

The semantic evaluation should compare:

- ranking results before semantic validation;
- final decisions after semantic validation;
- cases in which the top-ranked strategy is accepted;
- cases in which it is rejected or conditionally accepted;
- explanations generated through abduction, bonus, and contraction.

15.4 Performance and Blockchain Overhead

The prototype should measure the latency of:

- telemetry ingestion;
- asset extraction;
- vulnerability matching;
- OWL assertion generation;
- argumentation graph construction;
- graph storage;
- ranking;
- semantic validation;
- provenance recording;
- complete end-to-end execution.

The blockchain overhead should be estimated by comparing, where possible, an off-chain execution with the corresponding on-chain workflow. Existing monitoring and Caliper configurations may be reused, but the reported experiments must refer to the integrated thesis prototype.

15.5 Ablation and Comparative Analysis

The evaluation should include at least two comparisons:

- results obtained with and without runtime evidence;
- results obtained before and after semantic validation.

These comparisons are important for demonstrating the contribution of runtime observability and semantic reasoning rather than only the technical feasibility of blockchain execution.

16 Reproducibility and Expected Deliverables

The final prototype should provide a reproducible demonstration, preferably through a documented Docker-based procedure or a single orchestration command.

A complete execution should generate:

- observed software assets;
- matched vulnerabilities;
- generated OWL assertions;
- candidate mitigation strategies;
- the BWAF graph;
- the ranking stored on the ledger;
- the annotated graph and ranking explanation;

- the semantic validation result;
- the final recommended strategy;
- the corresponding provenance record.

The software repository should preserve a clear distinction between the baseline projects and the thesis implementation. The student should maintain:

- a tag or commit identifying the initial imported version;
- a dedicated thesis development branch;
- a commit history showing the introduced changes;
- a README describing installation and execution;
- configuration files and sample inputs;
- automated tests;
- scripts for reproducing the experiments;
- references to the original repositories, authors, and licenses.

A contribution matrix should be included in the thesis.

Component	**Baseline**	**Thesis contribution**
Telemetry processing	Agent-based CTI pipeline	Adaptation to the selected case study, extraction evaluation, and integration with the reasoning workflow.
CTI Knowledge Graph	Existing semantic asset and vulnerability modules	Formalization of the runtime-to-KG correlation and generation of contextual arguments.
Agent community	Logical decomposition in the proposal	Explicit implementation or rigorous mapping of agent roles, messages, and orchestration.
Argumentation graph	Initial graph generator and ranking chaincode	Formal rules for nodes, attack/support edges, weights, and explanation of the winner.
MCP integration	MCP-to-Fablo interaction	Completion of the store, rank, retrieve, validate, and provenance workflow.
Semantic validation	Tiny-ME semantic chaincode	Domain-level countermeasure validation and explanation of applicability or conflict.
Provenance	Ledger persistence primitives	PROV-O model covering evidence, activities, agents, decisions, and outcomes.
Evaluation	Existing tests and benchmark resources	End-to-end experiments, ablation studies, latency measurements, and qualitative analysis.

Table 1: Relationship between software baselines and original thesis contributions.

Before extending the architecture further, the implementation work should begin with a short technical assessment containing:

1. the current status of both repositories;
2. the missing functions with respect to the proposed architecture;

3. the components that will be implemented;

4. the planned experiments;

5. the explicit scope and exclusions of the thesis.

The main priority is to complete and validate the end-to-end reasoning path. A smaller but fully integrated prototype is scientifically preferable to a broader architecture in which OpenTelemetry, OSV, SBOM, multi-agent reasoning, blockchain, and autonomous mitigation are mentioned but not connected and evaluated in a reproducible workflow.

17 Case Study

A runtime OpenTelemetry event reveals that:

- `payment-api` uses `log4j-core 2.14.1`;
- the service is publicly exposed;
- deserialization activity is observed.

The Telemetry Agent generates runtime evidence.
The Vulnerability Agent queries the CTI Knowledge Graph and retrieves:

- CVE entries;
- CVSS severity;
- CWE categories;
- affected product information.

The Countermeasure Agent generates candidate strategies such as package upgrade, service isolation and deployment blocking.
The Strategic Agent aggregates all semantic assertions and constructs the BWAF graph.
The graph is serialized and submitted through MCP to the argumentation smart contract.
The generated BWAF contains free nodes representing contextual evidence such as CVE entries, runtime traces, exposed endpoints and vulnerability assertions, together with connected nodes representing candidate mitigation strategies.
For example:

- free node: detected CVE affecting log4j-core 2.14.1;
- free node: public runtime exposure;
- connected node: upgrade package;
- connected node: isolate service.

The Argumentation Smart Contract ranks candidate strategies according to attack/support relations derived from semantic assertions generated by the agents.
Finally, the Semantic Validation Smart Contract validates the selected mitigation through Tiny-ME reasoning, generates semantic explanations for the selected strategy, and stores provenance and reasoning information on-chain.